

Habla·ES — Project Specification & Build Handoff

Working title: Habla·ES (rename freely)

Status: Design + planning complete; runnable scaffold exists; ready to build.

This document is self-contained. It was written to be dropped into a fresh chat

(one with disk-edit / coding tools) with no prior context.

Everything needed to start

the actual work is here. A starter codebase (`spanish-tutor/`)

and three interactive

UI mockups accompany this spec in the handoff bundle.

0. How to use this document

You are picking up a project that has been designed but not yet built past a scaffold.

Read §1–§3 for the *what* and *why*, §4–§6 for the concrete technical contracts, §7 for the UI to build, §8 for decisions still open, and §10 for the immediate task list.

The accompanying files:

- `spanish-tutor/` — a Tauri v2 + Rust + React/TS scaffold (also `spanish-tutor.zip`). Compiles-pending; see §5.
- `ui-mockup.html` — mobile tab-based UI (chat + mechanics reference + bottom sheet).

- `ui-split-mockup.html` — split layout: chat half + live grammar breakdown half.
- `ui-assist-mockup.html` — the split layout **plus the assist slider** (the current target UI).

The **assist-mockup** is the closest thing to the intended product UI. Build toward it.

1. Vision & scope

A **free, open-source, local-first AI language tutor for adult self-learners**. It is

half conversation partner and half live grammar explainer. You talk (or tap) your way through a real conversation while a panel breaks down exactly how each sentence works.

Core properties:

- **Runs fully offline** on-device via **Gemma 4 E4B** (a ~4.5B-effective-param model in Google's Gemma 4 family, released 2026-04-02, designed for phones/edge). In 4-bit it needs ~5 GB RAM; a Pixel 9 Pro XL (16 GB, Tensor G4) runs it comfortably at ~10–25 tok/s.
- **Cross-platform via Tauri** (desktop first for dev; mobile is the eventual home).
- **Three target languages, Spanish first:** Spanish → Mandarin Chinese → Levantine Arabic. Only Spanish is in scope for the first build.
- **Evidence-based pedagogy is a hard requirement, not a nice-to-have** (see §2). Design decisions must trace to a second-language-acquisition rationale.

Non-goals (for now): accounts/cloud sync, gamified streaks, leaderboards, monetization.

2. Pedagogical foundations (these drive the design)

Every feature maps to a finding. Keep these in view when building.

1. **Comprehensible input / i+1 (Krashen).** Learners acquire language from input they *mostly* understand with a slight stretch. → Keep target language front and centre; reveal meaning on demand rather than pre-translating everything.
2. **Focus on form / just-in-time grammar.** Grammar is best noticed *in context*, when a structure actually appears — not as upfront rule lectures. → The breakdown panel and tap-to-notice highlights explain the sentence the learner is currently reading.
3. **Sheltering.** Constrain the tutor's output to the learner's known vocabulary + i+1.
This is the discipline human comprehensible-input teachers find hardest; automate it.
4. **High-frequency core.** Seed with the "Super 7" verbs (be, have, want, go, like, need — adapted per language) then the "Sweet 16." Deeper framing worth knowing:
Wierzbicka & Goddard's **semantic primes / Natural Semantic Metalanguage** — a ~65-item universal "basis set" of meanings. Useful as a north star for what a minimal core is.

5. **Chunking / lexical approach (Lewis).** Teach formulaic chunks ("me gustaría...") as whole retrievable units, not word-by-word.
 6. **Recasts + low affective filter.** Correct by gently re-modelling the right form ("you can also say..."), never a red X. Lower anxiety measurably raises willingness to communicate; AI chat is documented to reduce speaking anxiety.
 7. **Pushed but scaffolded output.** Encourage production with support (tappable starters, fill-in frames) that still requires the learner to choose and produce.
 8. **Spaced retrieval.** Resurface previously-met structures over time (FSRS/SM-2).
 9. **Adjustable, fadeable scaffolding.** Support should be dialable and should retreat as competence grows — the **assist slider** (§7.4).
-

3. Architecture

3.1 Core thesis — one skeleton, pluggable analyzers

Everything is shared across languages **except the analysis of the target language**. A

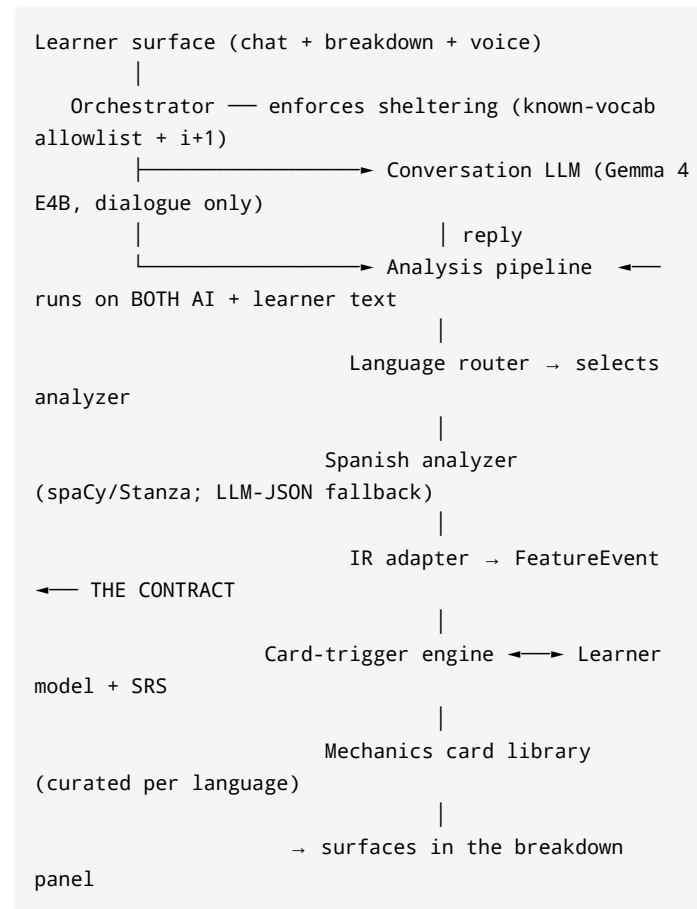
single pipeline runs every utterance; only the analyzer forks per language, and all

analyzers emit one common **intermediate representation (IR)** called `FeatureEvent`.

The card engine, learner model, orchestrator, and UI only ever read the IR — so adding

Mandarin or Levantine later means writing one new analyzer, changing nothing downstream.

3.2 Data flow



3.3 Why the three languages diverge (typology)

Only one band of the pipeline is language-specific because these three sit in different typological corners — this is the whole reason for the pluggable-analyzer design:

Language	Type	"Past tense" is...	Analyzer strategy	Tooling
Spanish	fusional	verb inflection (-é/-aba)	UD morphological features	spaCy es , Stanza
Mandarin	isolating	aspect particles (了/过), no inflection	segment → POS → constructions	jieba, Stanza, HanLP
Levantine Arabic	templatic + dialectal	root+pattern, clitics, diacritic-less	dialect-aware morph + LLM-assist	CAMEL Tools, SinaTools, Nâbra

Implication: the IR must carry **both** features[] (Spanish leans here) **and** constructions[] (Mandarin leans here). The Spanish → Mandarin build order is deliberate: doing Mandarin second forces the IR to generalise beyond inflection before Arabic (hardest) can demand a rewrite.

3.4 Build order

1. **Spanish** – best tooling; build the entire vertical slice (skeleton, IR, card engine, one analyzer).
2. **Mandarin** – stresses the IR (no inflection ⇒ constructions).
3. **Levantine** – hardest (dialect NLP, unstandardised orthography, weak ASR/TTS); LLM-assist + curated cards, once the core is proven.

Only step 1 (Spanish) is in scope for the first build.

4. Tech stack & key decisions

- **Shell:** Tauri v2. Rust backend, web frontend. Desktop for development; mobile is the goal.
 - **Frontend:** React + TypeScript + Vite.
 - **LLM:** local **Ollama** server (`http://localhost:11434`), model tag `gemma4:e4b` (⚠ verify with `ollama list` ; edit `MODEL_TAG` in `src-tauri/src/lib.rs` if different).
Behind an `LlmClient` trait (strategy pattern) so an embedded llama.cpp backend or a remote client can drop in later.
 - **Analyzer:** two implementations behind an `Analyzer` trait –
(a) `SpanishLlmAnalyzer` (default in scaffold): asks Gemma for strict-JSON morphology.
Fast to ship but this is the freeform-LLM-grammar approach that can drift.
(b) `SpacySidecarAnalyzer` (stub; the production path): a bundled Python sidecar running spaCy `es_core_news_md` (or Stanza) for deterministic Universal Dependencies.
Both emit identical `FeatureEvent` , so swapping is transparent. **Prioritise wiring (b).**
 - **Persistence:** learner state as JSON in the Tauri app-data dir (`learner.json`) in the scaffold. Upgrade to SQLite + an FSRS scheduler for real spaced repetition.
 - **Speech (later, out of first scope):** Whisper STT (and the fine-tuned `WhisperLevantineArabic` for Arabic); TTS is fine for es/zh, a known gap for Arabic dialect.
-

5. The existing scaffold (`spanish-tutor/`)

A runnable-after-fixups Tauri v2 project implementing the §3 pipeline as real modules.

5.1 File tree

```
spanish-tutor/  
├─ package.json, index.html, vite.config.ts,  
tsconfig.json  
├─ src/                                # React + TS frontend  
(BASIC chat+panel – upgrade to §7 UI)  
│   ├─ main.tsx, App.tsx, api.ts, types.ts,  
styles.css  
│   └─ components/ Chat.tsx, MechanicsPanel.tsx,  
Composer.tsx  
└─ src-tauri/  
    ├─ Cargo.toml, build.rs, tauri.conf.json  
    └─ cards/spanish.json          # 6 seed mechanics  
cards (real content)  
    └─ data/super7_es.json        # seed vocab  
allowlist  
    └─ src/  
        ├─ ir.rs                  # FeatureEvent +  
Token/Feature/Construction ← the contract  
        │   └─ llm/ mod.rs, ollama.rs    # LlmClient  
trait + Ollama backend  
        │   └─ analysis/ mod.rs, spanish.rs # Analyzer  
trait + Spanish (LLM impl + spaCy stub)  
        │   └─ orchestrator.rs          # sheltered turn loop  
(i+1 vocab gating)  
        │   └─ cards.rs                  # CardLibrary +  
trigger logic  
        │   └─ learner.rs                # LearnerModel + JSON  
persistence  
        │   └─ commands.rs              # Tauri commands:  
send_message, get_learner, reset_session  
        └─ lib.rs, main.rs              # wiring
```


5.2 Known caveats

- **Not yet compiled** in a Tauri toolchain — expect minor `cargo` fixups on first build.
The module graph, `include_str!` paths, and command registration are verified; types are not.
- The **frontend in the scaffold is the older simple chat+side-panel**, *not* the split + assist-slider design. The mockups (§7) are the real target; port them into React.
- Default analyzer is the LLM-JSON one (see §4). Move to the spaCy sidecar for reliability.

5.3 Run

```
ollama pull gemma4:e4b      # confirm tag with:
ollama list
ollama serve                 # usually already
running on :11434
npm install
npm run tauri dev
```

6. Data contracts (canonical)

These are the source of truth. Keep Rust and TS in sync.

6.1 IR — `FeatureEvent` (what every analyzer emits)

```
struct Token      { text: String, lemma: String,
pos: String, gloss: String } // pos = Universal POS
struct Feature    { key: String, value: String,
token_index: usize }        // e.g. Tense=Past ;
id() -> "Tense=Past"
struct Construction { id: String, token_span: (usize,
```

```

usize) } // e.g. "ser_vs_estar"
struct FeatureEvent {
    language: String, // "es"
    source_text: String,
    tokens: Vec<Token>,
    features: Vec<Feature>, // inflection-heavy
    langs fill this
    constructions: Vec<Construction>, //
    isolating/analytic langs fill this
}

```

6.2 Card + Trigger (curated content, cards/<lang>.json)

```

enum Trigger { Feature { key, value }, Construction {
id } } // serde tag="type", lowercase
struct Card {
    id, title, ceفر, // strings
    trigger: Trigger,
    explanation: String, // contextual "how it
works"
    example: String, // worked example
(target + gloss)
    contrast: String, // the "vs English"
aha
}

```

Card fires when its trigger is present in a `FeatureEvent` and it hasn't been shown recently;
engine prioritises the least-seen mechanic (i+1 pacing), caps at ~2 per turn.

6.3 LearnerModel (drives sheltering + novelty)

```

struct LearnerModel {
    level: String, // CEFR-ish,
e.g. "A1"
    known_vocab: HashSet<String>, // lemmas;
}

```

```

seeds the sheltering allowlist
  seen: HashMap<String, u32>,          // exposure
counts per feature/construction id
  recent_cards: VecDeque<String>,      // recency,
so cards don't repeat every turn
}

```

6.4 TurnResult (backend → frontend)

```

struct TurnResult {
  reply: String,
  analysis: FeatureEvent,
  cards: Vec<Card>,
  new_words: Vec<String>, // content lemmas above
known set = the i+1 introductions
}

```

7. UI / UX specification

Build target: `ui-assist-mockup.html` . The other two mockups show earlier/alternate

ideas worth keeping in the back pocket (a tab-based mobile layout with a mechanics-reference

tab and a bottom sheet). Port the mockups' HTML/CSS/JS behaviour into the React frontend.

7.1 Visual identity — deliberate two-layer system

The split isn't just layout; it encodes meaning. **Light paper = the language you're**

immersed in; dark steel = the analysis/scaffolding around it.

Design tokens (keep consistent across the whole product, including the architecture viz):

```

Dark shell (analysis): bg #0c1420 · chrome #14202e /
#1a2838 · inset #0c151f
                        line #26374a / #38506a · ink
#e8eef7 · muted #8a9bb3 · faint #5d6f88
Accents:                steel #6f9bff / deep #3f6ff5 ·
amber #e6b357 / deep #c78a2f (amber =
Spanish/mechanics)
Paper canvas (chat):    paper #f3f1ea / #eeeebe1 · line
#e0ddd2 · ink #1d242e · muted #6a7686
                        user bubble #e6ecfa on #24405f
Type: Space Grotesk (display/wordmark) · IBM Plex
Sans (UI) · IBM Plex Mono (linguistic tags/data)
        Newsreader serif (the TARGET-LANGUAGE reading
text – its own dignified voice)
Per-language accent (for the 3-lang architecture viz):
es amber #e7b24c · zh jade #3fc9a6 · ar rose #ec6d9c

```

7.2 Layout — split screen (the current direction)

- **Half chat, half live breakdown.** Side-by-side on wide screens (chat left, breakdown right); stacked on a phone (chat top, breakdown bottom). ~50/50 (make the ratio adjustable; consider a collapsible breakdown drawer on phone since 50% is tight there).
- **Chat half:** warm paper canvas, Spanish in serif, tutor bubbles white, user bubbles steel-tint. Recast notes appear as gentle steel-tinted notes under the learner's message (no red X).
- **Breakdown half (of the latest turn, dark):** four stacked sections —
 1. the sentence (notable forms highlighted),
 2. **word-by-word interlinear gloss** (Spanish / English / POS),
 3. **features detected** (mono chips: Tense=Past , Person=1 , ...),

4. **what's happening** — the explainer card(s) for the mechanic(s), with the vs-English contrast.

- **Re-pin:** tapping any tutor message focuses the breakdown on that turn (marker shows which).

Default focus = latest turn. New turns auto-advance the breakdown.

7.3 Tap-to-notice

Notable grammatical forms carry a faint amber dotted underline in the actual message.

Tapping focuses/scrolls the breakdown to that word's mechanic.

Grammar is *pull*, contextual,

tied to the exact word — never an upfront lecture. (Open question: optional first-encounter

auto-peek so beginners know which words reward attention — see §8.)

7.4 The Assist slider (key feature)

A 4-stop slider that **fades scaffolding in/out**, so a near-total beginner can bootstrap

into a real conversation and dial the help down as they gain

footing. It governs the

conversation half only — the breakdown half stays fully detailed at every level (it's a reference layer, not a crutch).

Stop	Name	Tutor line shows	Reply scaffolds
3	Full support	full English translation under the bubble + per-word gloss	complete, tap-to- send replies (hold a convo by tapping)

Stop	Name	Tutor line shows	Reply scaffolds
2	Guided	per-word English gloss (interlinear, in-bubble)	fill-in-the-blank frames (Yo compré ____) inserted into composer
1	Light	key words underlined only	a couple of short starters
0	Immersion	Spanish only, nothing revealed	none — tap a word if stuck; free typing

Future: **auto-fade** (slider drifts left as exposure counts climb) and possibly

asymmetric assist (separate "understanding" vs "speaking" sliders).

7.5 Other elements (from earlier mockups, keep)

- **Mechanics reference view/tab**: a browsable library of every structure met, each with an exposure ring + seen-count (spaced-retrieval surface). Calm progress, no streaks.
- **Scaffold help chip** ¿Cómo digo...? — help without switching to English.
- **Voice** (later): low-stakes mic input.
- **Calm competence indicators** in the top bar (level chip, "N met") — no gamification.

8. Open decisions (resolve during build)

1. **IR schema:** strict Universal Dependencies vs a looser custom schema. UD is clean for es/zh but fights Levantine's root-and-pattern morphology.
 2. **Analyzer priority:** how soon to move Spanish off LLM-JSON onto the spaCy sidecar (recommended: early). For Arabic later: CAMEL-primary + LLM-fallback, or the inverse.
 3. **Reading canvas:** keep the light paper chat, or offer a single dark theme (OLED/identity).
 4. **Grammar push vs pull:** current design is pull (tap to learn). Add an optional first-encounter auto-peek so new structures announce themselves once?
 5. **Assist auto-fade** and **asymmetric assist** — build now or later?
 6. **Breakdown target:** currently breaks down the *tutor's* turn. Add a "your turn" toggle to break down (and correct) the *learner's* message?
 7. **Phone split:** fixed 50/50 vs collapsible/drag-resizable breakdown drawer; split ratio.
 8. **SRS:** integrate FSRS + move persistence to SQLite.
-

9. Prior art & reuse

- **skellysubs** (freemocap, python-only branch) — the author's earlier project, directly reusable: an `ai_clients/` **strategy layer** (Ollama / OpenAI / HuggingFace), Whisper transcription, a **word-level translation + romanization** pipeline, and sample data already translated into **Spanish, Levantine Arabic, and Mandarin** — the same three targets. Lift

the ai-client abstraction and the word-level gloss/romanization ideas.

- **Reference FOSS apps:** *freelingo* (self-hosted, Ollama, SM-2 SRS, Kokoro TTS + faster-whisper) and *Discute* (speaking practice: Whisper + LLM + Kokoro). Don't rebuild SRS/TTS/STT – those are solved.

10. Immediate next steps (for the new chat)

1. **Get the scaffold compiling** – `cargo build` in `src-tauri`, fix minor type/API issues.
2. **Confirm the model tag** – `ollama list`; set `MODEL_TAG` in `lib.rs`.
3. **Smoke-test the loop** – run `npm run tauri dev`, send a message, confirm reply + `FeatureEvent` + cards flow back.
4. **Port the target UI** – replace the scaffold's basic frontend with the **split + assist-slider** design from `ui-assist-mockup.html` (React components: Chat, BreakdownPanel, AssistSlider, Composer/Scaffolds). Keep the two-layer identity and design tokens (§7.1).
5. **Wire the assist slider to real behaviour** – it should shape the orchestrator prompt (sheltering intensity, whether to include an English translation line) and the scaffold set.
6. **Replace the LLM analyzer with the spaCy sidecar** for deterministic Spanish morphology (§4/§5); keep the LLM analyzer as fallback.
7. **Expand the Spanish card library** beyond the 6 seeds; ensure triggers cover the A1–A2 core.
8. Then iterate on §8 decisions.

End of spec. Bring `SPEC.md`, `spanish-tutor/` (or `.zip`), and the three `ui-*.html` mockups into the new chat.